

Design Doc: Agentic-Driven Reorg Case

1. Executive Summary

This system turns freeform reorganization requests (email, Slack, documents) into an inspectable, governed **Reorg Case**: structured interpretation, authoritative validation, dependency-aware planning, human approval, mixed automated/manual execution, and reconciliation against systems of record.

Business outcome: reorgs stop living as tribal checklists in people’s heads. Operators can see what was requested, what was verified, what is blocked, what a human must key in, and whether downstream state actually matches the approved plan—before finance reports surface errors weeks later.

Central principle: Agents interpret and plan. Policies authorize. Tools execute. Humans resolve ambiguity and high-impact decisions. The system verifies the result.

2. Problem

Companies reorganize often, but change does not flow automatically between HRIS, headcount planning, cost allocation, GL mapping, and reporting. Each system is updated by hand. Order of operations is critical and uncoded. Runbooks drift. Approvals are asserted in prose (“Finance approved”) without independent verification. At least one downstream system may have no API. Sensitive PII and compensation data are in play.

The deeper failure mode is not “manual work is slow”—it is that **there is no operational source of truth for whether a reorg completed successfully**. Failures remain invisible until reporting breaks later.

3. Goals

- Transform unstructured reorg intent into an explicit, typed change set
 - Validate claims against authoritative enterprise data (not against the LLM’s confidence)
 - Coordinate ordered cross-system execution with explicit dependencies
 - Support automated adapters and first-class human-mediated steps (no-API systems)
 - Enforce deterministic approval before high-impact writes
 - Detect inconsistency via reconciliation before reporting propagation
 - Make workflow state understandable to HR, FP&A, Legal, Finance, and Ops
 - Maintain an auditable record of interpretation, decisions, and mutations
-

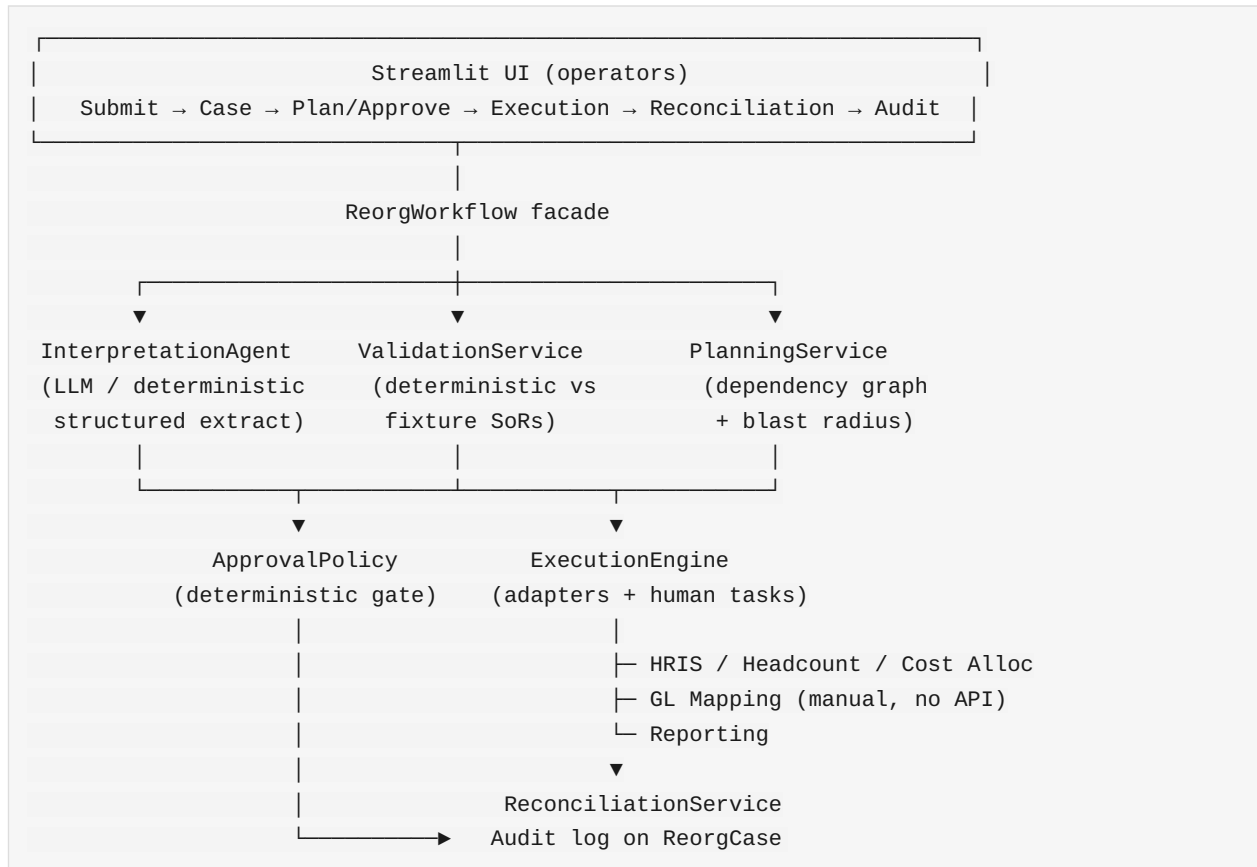
4. Non-Goals

- Real Slack/email ingestion or production company-system integrations
 - Production identity, SSO, or fine-grained RBAC
 - Durable workflow engines (Temporal/Cadence), retries at scale, or multi-region HA
 - Full compensation-change workflows
 - Autonomous multi-agent bargaining across systems
 - Polished production design system / mobile app
 - Claiming measured “hours spent” as a quality signal
-

5. Design Principles

1. **Domain systems remain authoritative** for employee, budget, and financial data.
 2. **The Reorg Case is authoritative for workflow execution**—not for HR or ledger truth.
 3. **LLM output is proposed interpretation, not truth.**
 4. **Agents reason; policies authorize; tools execute.**
 5. **Ambiguity and unverified claims stop unsafe propagation** (or force human gates).
 6. **Humans provide judgment and actuate no-API steps—they are not the orchestrator.**
 7. **Execution must be reconciled** (expected vs observed), not trusted via HTTP 200 or “Mark Complete.”
 8. **Every significant decision and mutation is auditable**, with PII minimized in logs/prompts.
-

6. Architecture



Layers stay thin: UI calls the workflow facade; business rules do not live in Streamlit callbacks.

7. Reorg Case Data Model

The Reorg Case serves as the system of record for reorg execution, while domain systems such as HRIS and Finance remain authoritative for their underlying data.

It answers: what was requested; what source produced it; what the system interpreted; what was independently validated; what assumptions/ambiguities remain; what changes are proposed and in what order; which approvals are required vs obtained; what executed or failed; what needs a human; whether observed state matches expected state; whether the reorg is actually complete.

Typed Pydantic models (`ExtractedRequest` , `ValidationResult` , `ChangePlan` , `HumanTask` , `ReconciliationResult` , `AuditEvent`) prevent loosely typed dictionaries from becoming the integration contract.

8. End-to-End Workflow

Capture → **Interpret** → **Validate** → **Plan** → **Approve** → **Execute** → **Reconcile**

1. **Capture** — preserve raw freeform text and source type.
 2. **Interpret** — LLM (or demo extractor) emits schema-validated structure. Claimed approvals are flagged, never auto-verified.
 3. **Validate** — deterministic checks against HRIS/org/cost center/approval fixtures: entity existence, headcount, eligibility, inactive CCs, exception membership, independent approval lookup.
 4. **Plan** — build actions with `depends_on` (not a flat numbered list).
 5. **Approve** — policy layer requires HR Ops (+ Finance when cost centers move). LLM cannot grant this.
 6. **Execute** — automated adapters mutate simulated systems; GL mapping becomes a structured `HumanTask`.
 7. **Reconcile** — compare expected vs observed; block reporting if GL mismatches; set case terminal status accordingly.
-

9. Agentic Building Blocks

Step	Mechanism	Why
Interpretation	LLM structured extraction (optional live model via <code>LLM_API_KEY</code> env; deterministic demo provider otherwise). Used only to parse freeform intake.	Intake is inherently unstructured
Schema boundary	Pydantic validation	Prevent freeform model text from driving mutations
Validation	Deterministic services vs SoR fixtures	Truth must not be probabilistic
Planning	Deterministic dependency graph	Order is a business invariant
Authorization	Deterministic <code>ApprovalPolicy</code>	Separation of reasoning and authority
Execution	System adapters + human tasks	Tools execute; humans actuate no-API steps
Reconciliation	Deterministic compare	“Done” means state matches plan

No swarm of per-system agents: agency adds little once the change set is structured, and it obscures the single operational case record.

10. Human-in-the-Loop Strategy

Humans are required when:

- Cost center / financial impact changes (Finance approval)
- Org/manager HRIS writes (HR Ops approval)
- Unverified approval claims appear in source text
- Validation conflicts, ambiguities, low confidence, or large blast radius
- A target system has **no API** (GL mapping)—modeled as a first-class task with instructions, required fields, expected state, and completion evidence

Humans are **not** required to remember the global sequence: the case owns orchestration.

11. Security / PII

Prototype controls:

- Prompt redaction heuristics (emails, compensation mentions)
- Audit metadata avoids raw sensitive payloads
- Explicit approval before writes
- Clear separation of LLM permissions vs execution adapters

Production would additionally require: SSO + RBAC/SoD, DLP classification, field-level encryption, secrets management, private model endpoints or strict egress, retention policies, and legal hold alignment. Systems of record remain authoritative; the case store is an operational record with minimized copies of PII.

12. Alternatives Considered

Pure deterministic workflow / form-based intake

Strong for known templates. **Rejected as the primary intake** because initiating requests arrive as freeform prose and vary by reorg. Forms remain a valuable *downstream* confirmation UI after extraction.

Fully autonomous multi-agent execution

Attractive demo narrative. **Rejected** because high-impact enterprise mutations need deterministic policy, approval, dependency enforcement, and reconciliation. Autonomy without measurement increases blast radius.

Independent agent per downstream system

Scales organizationally for large integration estates. **Deferred** for this prototype: coordination complexity obscures the Reorg Case as the single operational SoR for execution, which is the product insight we need to demonstrate.

13. Risks and Failure Modes

Incorrect extraction from ambiguous source text

- **Blast radius:** Wrong people/CC moved if trusted blindly.
- **Detection:** Authoritative validation, confidence, ambiguity surfacing, human review.
- **Mitigation:** LLM never writes; conflicts block or force approval; evaluation suite before increasing autonomy.

Partial execution across systems

- **Blast radius:** HRIS updated, finance systems stale → reporting lies.
- **Detection:** Action statuses + dependency blocking; reconciliation.
- **Mitigation:** Dependents do not run on failed prerequisites; terminal case status ≠ complete until reconcile passes; compensating actions as a next step.

Human marks manual GL task complete incorrectly

- **Blast radius:** Silent finance misallocation until month-end.
 - **Detection:** Reconciliation of observed GL vs approved expected CC; reporting blocked on mismatch.
 - **Mitigation:** “Mark Complete” is necessary but not sufficient; require evidence and SoR readback in production.
-

14. Assumptions and Open Questions

Assumptions

- Fixture directory/HRIS/cost centers are sufficient stand-ins for demo SoRs.
- Finance approval authority is represented by an approvals ledger (not email prose).
- GL mapping has no API and is owned by Finance Operations.
- Org moves that include cost center changes always need Finance + HR Ops approval in this policy pack.
- Effective date is carried as a field but not calendared into a job scheduler in the prototype.

Open questions

- What system records legally authoritative approvals?
 - Which change classes require Legal vs HR vs Finance vs both?
 - Are downstream writes idempotent? Can they be reversed?
 - Which system owns the canonical effective date?
 - Expected reorg volume, blast-radius distribution, and SLAs for manual steps?
 - What evidence standard is required for manual completion?
-

15. Prototype Scope

Built

- Typed Reorg Case + audit trail
- Slice A: freeform → structured extraction → authoritative validation (incl. unverified approval claims)
- Slice B: dependency-aware plan, policy approval gate, simulated adapters, manual GL task, reconciliation
- Streamlit operator UI for the live walkthrough
- Two demo scenarios (success path + ambiguity/failure path)
- Focused pytest suite encoding design intent

Deliberately not built

Real Slack/email ingestion; real production system integrations; production auth/RBAC; durable workflow infra; production secrets management; full compensation workflows; comprehensive eval platform; production observability; polished production UI.

I concentrated the prototype on the areas containing the most important technical and product judgment: converting ambiguous intent into a validated change plan and safely coordinating automated and human-mediated execution.

16. AI Usage and Human Overrides

- AI suggested direct autonomous propagation after extraction → **overridden**: execution authorization remains a deterministic policy gate.
- AI suggested modeling each integration as an independent agent → **simplified** to deterministic adapters; the Reorg Case stays the coordination SoR.
- AI suggested treating “Finance has approved” as sufficient → **rejected**: claimed approval must be independently verified; otherwise surface warning and require Finance approval.

- AI helped draft candidate schemas → **manually simplified** so the live demo remains explainable in minutes.
 - AI-shaped copy for runbook-like manual task instructions → **kept**, because the product insight is structured human actuation, not removing humans.
-

17. Next Steps / With More Time

1. **Validate workflow with domain owners** (HR, FP&A, Legal, Finance, Ops)—largest risk is incorrectly codifying tribal process.
2. **Connect authoritative systems** with versioned contracts replacing fixtures.
3. **Build an evaluation suite** of historical/synthetic reorgs (happy path, ambiguity, contradictions, missing approvals, bad CCs, partial failures, large blast radius). Measure extraction accuracy, planning correctness, ambiguity detection, unsafe-action rate. Do not increase autonomy before measuring it.
4. **Harden policy and authorization** (PII, compensation, blast radius, confidence, SoD).
5. **Durable orchestration** — persisted state, retries, idempotency, resumability, timeouts, compensating actions.
6. **Productionize manual tasks** — work-management tooling + evidence + mandatory reconciliation.
7. **Business-facing operational UI** with statuses: Proposed, Needs Review, Approved, Executing, Blocked, Needs Human Action, Reconciling, Completed.